



香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

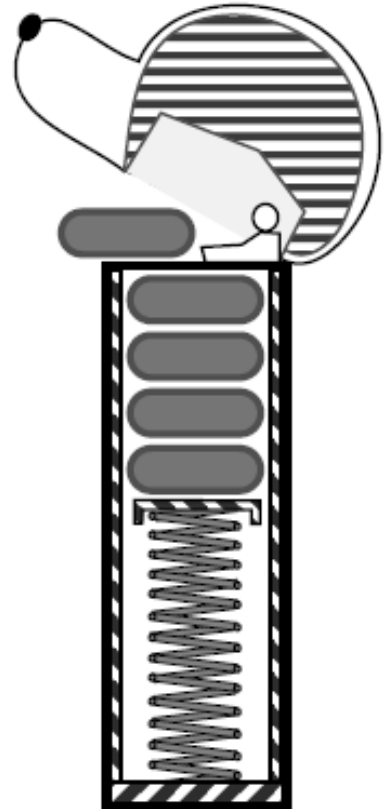
Introduction to Computer Engineering: Programming Applications

Lecture 9 Data Structure and Algorithms Part II Stack and Queue

**Prof. Junhua Zhao
School of Science and Engineering**

Stack

- A **stack** is a collection of objects that are inserted and removed according to the **last-in, first-out (LIFO)** principle
- A user may **insert** objects into a stack **at any time**, but may only access or remove the most recently inserted object that remains (**at the so-called “top” of the stack**)



Example: Web Browser

- Internet Web browsers store the addresses of recently visited sites in a stack. Each time a user visits a new site, that site's address is “pushed” onto the stack of addresses. The browser then allows the user to “pop” back to previously visited sites using the “back” button.

♥ Elon Musk liked



DogeDesigner  @cb_doge · 16h

...

In 1995, Elon Musk applied for a job at Netscape, sent his resume, but he was too shy to talk to anyone. So he decided to start his own company (Zip2).

Imagine if [@elonmusk](#) had got his dream job at Netscape. 🐶



Elon Musk and 2 others



401



539



6,103



1M



Example: Text editor

store edit history to stack (บันทึกการแก้ไข save ไว้ใน stack) remove top element from text file

- Text editors usually provide an “undo” mechanism that cancels recent editing operations and reverts to former states of a document. This undo operation can be accomplished by keeping text changes in a stack.

The stack class

↳ just follow the principle

- Generally, a stack may contain the following methods:

recommended

S.push(e): Add element e to the top of stack S.

S.pop(): Remove and return the top element from the stack S; an error occurs if the stack is empty.

S.top(): Return a reference to the top element of stack S, without removing it; an error occurs if the stack is empty.

S.is_empty(): Return True if stack S does not contain any elements.

len(S): Return the number of elements in stack S; in Python, we implement this with the special method `__len__`.

The Code of Stack Class

implemented based on list

Important difference:
We can access the value of any data in list
modify
(In stack we can access only the top element)

```
class ListStack:

    def __init__(self):
        self.__data = list()  # save all data
                               # ↳ empty list (stack)

    def __len__(self):
        return len(self.__data)

    def is_empty(self):
        # True when 0, False when
        return len(self.__data) == 0

    def push(self, e):
        # insert into list
        self.__data.append(e)

    def top(self):
        if self.is_empty():
            print('The stack is empty.')
        else:
            return self.__data[self.__len__() - 1]

    def pop(self):
        if self.is_empty():
            print('The stack is empty.')
        else:
            # remove last element from the list
            return self.__data.pop()
```

S = ListStack()
S.push(10)
→ S.__data = [10]
S.push(20)
→ S.__data = [10, 20]

The code to use stack class

```
def main():  
    s = ListStack()  
    print('The stack is empty? ', s.is_empty())  
    s.push(100)  
    s.push(200)  
    s.push(300)  
    print(s.top()) 300  
    print(s.pop()) 300  
    print(s.top()) 200
```


Practice: Reverse a list using stack

- Write a program to reverse the order of a list of numbers using the stack class

Solution:

```
from stack import ListStack

def reverse_data(oldList):
    s = ListStack()
    newList = list()

    for i in oldList:
        s.push(i)  # ចូលទៅក្នុង stack ពី oldList ទៅ s (ក្នុង stack)
                   # (order ប្រែប្រួល list មិន)

    while (not s.is_empty()):
        mid = s.pop()  # យកចេញពី stack ទៅ list
                     # (LIFO)
        newList.append(mid)

    return newList

def main():
    oldList = [1, 2, 3, 4, 5]
    newList = reverse_data(oldList)
    print(newList)
```

Practice: Brackets match checking

- In correct arithmetic expressions, the opening brackets must match the corresponding closing brackets. Write a program to check whether all the opening brackets have matched closing brackets.

([{ = opening bracket
)] } = closing bracket

Solution:

```
from stack import ListStack

def is_matched(expr):
    lefty = '([{'  # matching order
    righty = ')]}'

    s = ListStack()

    for c in expr:
        # check if c is opening bracket
        if c in lefty:
            s.push(c)  # push into stack
        # ignore all characters aside from the bracket
        elif c in righty:
            if s.is_empty():
                # ต้องตรวจสอบว่ามีอะไรใส่เข้าไปใน s หรือไม่
                # False ถ้า s ว่าง (ไม่พบวงเล็บเปิด)
                return False
            # *str*
            if righty.index(c) != lefty.index(s.pop()):
                # check opening bracket's index in lefty
                # pop stack and compare
                return False
            # ถ้า char ที่ออกมาตรงกับใน str then
            # ถ้าไม่ = return index

    return s.is_empty()

def main():
    expr = '1+2*(3+4)-[5-6]'
    print(is_matched(expr))
    expr = '(()))}]'
    print(is_matched(expr))
```

ใช้ว่าเป็นวงเล็บเปิด-ปิด type เดียวกันไหม (เครื่องหมาย [] ไม่ใช้)

ต้องตรวจสอบว่ามีอะไรใส่เข้าไปใน s หรือไม่ False ถ้า s ว่าง (ไม่พบวงเล็บเปิด)

str

check opening bracket's index in lefty

pop stack and compare

ถ้า char ที่ออกมาตรงกับใน str then

ถ้าไม่ = return index

Practice: Matching Tags in HTML Language

- HTML is the standard format for hyperlinked documents on the Internet
- In an HTML document, portions of text are delimited by HTML tags. A simple opening HTML tag has the form “<name>” and the corresponding closing tag has the form “</name>”

HTML Tags

- Commonly used HTML tags that are used in this example include
 - `body`: document body
 - `h1`: section header
 - `center`: center justify
 - `p`: paragraph
 - `ol`: numbered (ordered) list
 - `li`: list item

An example of HTML document

change from
check type of bracket
to
check tag
(substring, not single char)

```
<body>
<center>
<h1> The Little Boat </h1>
</center>
<p> The storm tossed the little
boat like a cheap sneaker in an
old washing machine. The three
drunken fishermen were used to
such treatment, of course, but
not the tree salesman, who even as
a stowaway now felt that he
had overpaid for the voyage. </p>
<ol>
<li> Will the salesman die? </li>
<li> What color is the boat? </li>
<li> And what about Naomi? </li>
</ol>
</body>
```

(a)

The Little Boat

The storm tossed the little boat like a cheap sneaker in an old washing machine. The three drunken fishermen were used to such treatment, of course, but not the tree salesman, who even as a stowaway now felt that he had overpaid for the voyage.

1. Will the salesman die?
2. What color is the boat?
3. And what about Naomi?

(b)

Solution:

```
from stack import ListStack
```

```
def is_matched_html(raw):  
    s = ListStack()  
    j = raw.find('<')
```

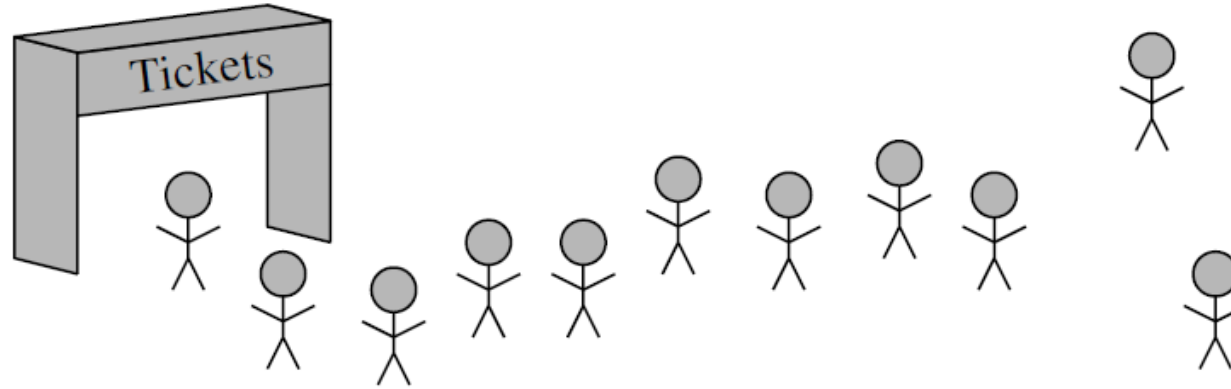
```
    while j != -1: We found <  
        return -1 k = raw.find('>', j+1) return index  
        if k == -1: incomplete bracket index j+1 မရှိပါ  
            return False  
        tag = raw[j+1:k] slicing <> မှန်  
        if not tag.startswith('/'): check whether it is opening tag or closing tag  
            opening s.push(tag)  
        else:  
            closing if s.is_empty(): ခွဲအိတ် opening tag မှန်  
                return False  
            if str tag[1:] != s.pop(): closing tag (စာ / စာ) မှန် tag မှန် opening tag မှန်  
                closing tag return False opening tag  
            j = raw.find('<', k+1) find location of next tag, (value of j changed)  
    return s.is_empty()
```

```
def main():  
    fhand = open('sampleHTML.txt', 'r')  
    raw = fhand.read() read and save inside str 'raw'  
    print(raw)  
    print(is_matched_html(raw))
```

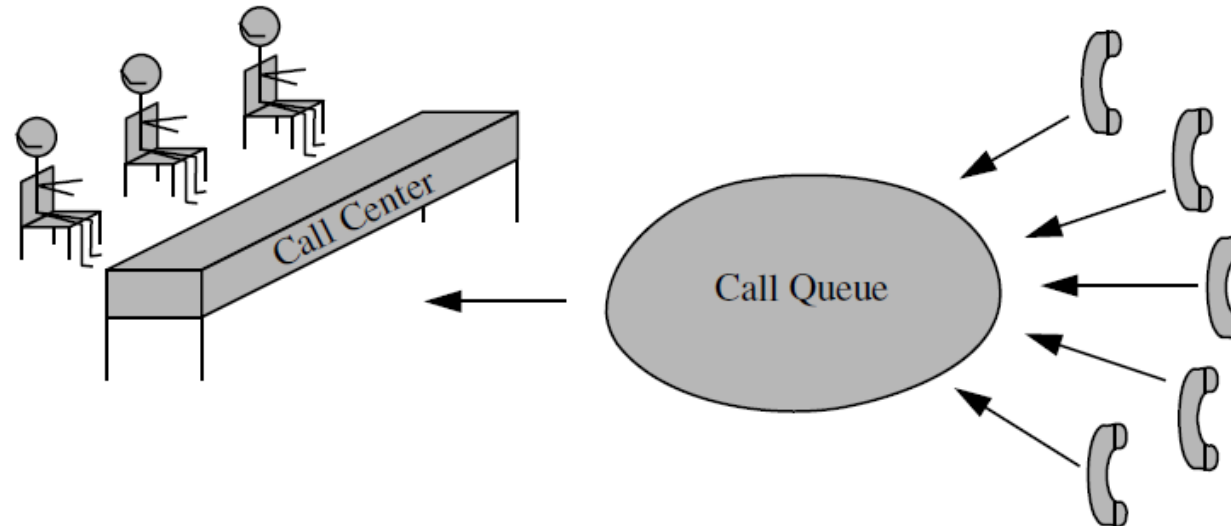

Queue

- **Queue** is another fundamental data structure
- A queue is a collection of objects that are inserted and removed according to the **first-in, first-out (FIFO)** principle
LIFO last in, last out
- Elements can be inserted **at any time**, but only the element that has been in the queue **the longest** can be next removed

Applications of Queue



(a)



(b)

The queue class

- The queue class may contain the following methods:

push

Q.enqueue(e): Add element e to the back of queue Q.

pop

Q.dequeue(): Remove and return the first element from queue Q; an error occurs if the queue is empty.

top

Q.first(): Return a reference to the element at the ^(first) front of queue Q, without removing it; an error occurs if the queue is empty.

Q.is_empty(): Return True if queue Q does not contain any elements.

len(Q): Return the number of elements in queue Q; in Python, we implement this with the special method `__len__`.

The code of queue class

```
class ListQueue:
    default_capacity = 5 Not data fields

    def __init__(self): list containing 5 None
        self.__data = [None]*ListQueue.default_capacity
        self.__size = 0 Static data field
        self.__front = 0 How many data we can save in this queue = 5 (limit)
        self.__end = 0

    def __len__(self):
        return self.__size

    def is_empty(self): empty queue
        return self.__size == 0

    def first(self):
        if self.is_empty():
            print('Queue is empty.')
        else:
            return self.__data[self.__front]

    def dequeue(self):
        if self.is_empty():
            print('Queue is empty.')
            return None

        answer = self.__data[self.__front] = 10
        self.__data[self.__front] = None เราคือเราออก เปลี่ยนเป็น none
        self.__front = (self.__front+1) \
            % ListQueue.default_capacity
        self.__size -= 1
        return answer q = ListQueue()

    def enqueue(self, e): q.enqueue(10) → เติม 10 เข้า queue
        if self.__size == ListQueue.default_capacity:
            print('The queue is full.')
            return None
        self.__data[self.__end] = e ใส่ index ที่ว่างใน queue
        self.__end = (self.__end+1) \
            % ListQueue.default_capacity work in circular way
        self.__size += 1 end = 4 → เปลี่ยนเป็น 0 (ตัวที่ 5)

    def outputQ(self):
        print(self.__data)
```

*ทำแบบนี้เราต้องทำไปเรื่อยๆ
มันจะกินขนาดของพื้นที่ตัวแรก/ตัวออกที่ตัวแรก*

*ถ้าคิวเต็มแล้ว
เติม=เติมไม่ได้*

*q.enqueue(20)
q.enqueue(30)
q.enqueue(40)
q.enqueue(50)
q.enqueue(60) → ทำไปไม่ได้ ก็คือมันกลับ
return none*

Practice: Chemistry Reaction Simulation

- Write a program simulates a chemical reactor where molecules are sequentially added to a reaction chamber and processed in a First-In, First-Out (FIFO) manner. This models how reactants enter a reactor and undergo reactions one by one, reflecting the queue's behavior.

Chemistry Reaction Simulation

```
# Chemistry function to simulate a reaction
def simulate_reaction(molecule):
    """Simulate a chemical reaction based on molecule type."""
    reaction_products = {
        "H2": "H2O (with O2)",          # Hydrogen reacts with oxygen to form water
        "O2": "O3 (with electricity)",   # Oxygen can form ozone under specific conditions
        "N2": "NH3 (with H2)",           # Nitrogen and hydrogen form ammonia
        "CO2": "C6H12O6 (with H2O)"      # Photosynthesis-like reaction (simplified)
    }
    return reaction_products.get(molecule, "No reaction")
    # input key return corresponding value in case key not in dict

# Example of using the queue for chemical reactions
def chemistry_queue_example():
    # Create a queue to simulate molecules entering the reaction chamber
    molecule_queue = Queue()

    # List of molecules entering the chamber in sequence
    molecules = ["H2", "O2", "N2", "CO2", "H2", "O2"]

    # Enqueue each molecule into the queue
    for molecule in molecules:
        molecule_queue.enqueue(molecule)  # enqueue molecules into molecule_queue
        print(f"Molecule {molecule} added to the queue.")

    print("\nProcessing molecules in the reaction chamber (FIFO order):")

    # Process each molecule in FIFO order
    while not molecule_queue.is_empty():
        molecule = molecule_queue.dequeue()  # dequeue molecule from queue
        reaction_result = simulate_reaction(molecule)
        print(f"Molecule {molecule} reacted to form: {reaction_result}")

# Run the chemistry queue example
chemistry_queue_example()
```

Bubble sort

Sorting = rearrange items in order we want (from list of random order)
Binary search

- **Bubble sort** is a simple sorting algorithm

useful

- Its general procedure is:

- 1) Iterate over a list of numbers, compare every element i with the following element $i+1$, and swap them if i is larger
- 2) Iterate over the list again and repeat the procedure in step 1, but ignore the last element in the list
- 3) Continuously iterate over the list, but each time ignore one more element at the tail of the list, until there is only one element left

Practice: Bubble sort over a standard list

The program: $O(n^2)$

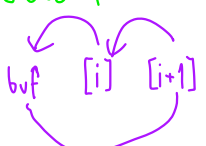
```
def bubble(bubbleList):  
    listLength = len(bubbleList)  
    while listLength > 0:  
        for i in range(listLength - 1):  
            if bubbleList[i] > bubbleList[i+1]:  
                buf = bubbleList[i]  
                bubbleList[i] = bubbleList[i+1]  
                bubbleList[i+1] = buf  
            listLength -= 1  
    return bubbleList
```

$O(1)$ { nothing to do with problem size

$0, \dots, \text{listLength}-2$

listLength = 1 1 ครั้ง

สลับ [i], [i+1] ถ้า [i] > [i+1]



```
def main():  
    bubbleList = [3, 4, 1, 2, 5, 8, 0, 100, 17]  
    print(bubble(bubbleList))
```

3	4	1	2	5	8	0	100	17
1	2	3	4	0	5	8	17	100
⋮								
0	1	2	3	4	5	8	17	100

ค่าของ 0 มาจาก 0

Quick sort

- Quick sort is a widely used algorithm, which is more efficient than bubble sort
- The main procedure of quick sort algorithm is:
 - 1) Pick an element, called a **pivot**, from the array
 - 2) **Partitioning**: reorder the array so that all elements with values less than the pivot come before the pivot, while all elements with values greater than the pivot come after it (equal values can go either way). After this partitioning, the pivot is in its final position. This is called the **partition operation**
 - 3) Recursively apply the above steps to the sub-array of elements with smaller values and separately to the sub-array of elements with greater values

Practice: Quick sort over a standard list

```
def quickSort(L, low, high):
```

```
    i = low
```

```
    j = high
```

```
    if i >= j: Nothing left in range
```

```
        return L
```

```
    pivot = L[i]
```

```
    save save
```

```
    while i < j:
```

```
        while i < j and L[j] >= pivot:
```

```
            j = j - 1
```

*เอาตัวน้อยกว่า
ไปอยู่หน้าตัว*

```
        L[i] = L[j]
```

*เอาตัวมากกว่า
ไปอยู่หน้าตัว*

```
        while i < j and L[i] <= pivot:
```

```
            i = i + 1
```

*เอาตัวมากกว่า
ไปอยู่หน้าตัว*

```
        L[j] = L[i]
```

```
    L[i] = pivot
```

```
    quickSort(L, low, i-1)
```

```
    quickSort(L, j+1, high)
```

```
    return L
```

```
L = [6, 5, 3, 10, 12, 2, 4]
```

```
print(quickSort(L, 0, 6))
```

$L = [4, 5, 3, 2, 6, 12, 10]$ Pivot = 6

—

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

—

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

—

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

—

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

—

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

—

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

—

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

—

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

—

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

—

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

—

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

—

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

—

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j

i

j